# CC3092 · Laboratorio 9 — Series de tiempo y forecasting

**Javier López (23415) · Javier Cifuentes (23079)** · Universidad del Valle de Guatemala · noviembre 2026 · Panel: 100 clientes de ElectricityLoadDiagrams20112014, H = 24 h, 92 orígenes de prueba

## 1. Datos, preparación y auditoría de fugas

El archivo crudo (140,256 × 370) registra potencia media en kW cada 15 min. La energía horaria es **kWh = Σ₄ kW·0.25 h = (x₁+…+x₄)/4**. La marca de tiempo es el *inicio* del intervalo (el bloque anómalo del cambio de horario está en 01:00–01:45). **Cambio de horario:** en marzo la hora 01:00 viene en cero (suma del panel ≈ 4k frente a ≈ 120k kWh en las horas vecinas) y se interpola; en octubre la hora 01:00 contiene 2 h (proporción 1.5–1.7 frente a la hora previa) y se divide entre 2. La grilla queda regular de 24 h/día. **Filtro:** 317 clientes con consumo > 0 desde el 2012-01-01, sin NaN y sin tramos en cero ≥ 7 días. Cada serie tiene 26,304 horas (panel filtrado: 8.34 M observaciones). Por cómputo (CPU de 2 núcleos, sin GPU) se usan **100 clientes** (semilla 42; 2.63 M observaciones), los mismos para todos los modelos. **Escalas:** el consumo medio va de 4.3 a 48,818 kWh (mediana 148) en los 317, y de 8.3 a 11,480 kWh en los 100. Se normaliza con **z-score por serie usando μ, σ de entrenamiento**; la normalización global, sin normalización y RevIN se comparan en §4. **Patrones:** mínimo de 0.62× la media a las 04:00, pico de 1.29× a las 19–20 h, y poca diferencia entre días de la semana en el agregado. En los feriados (`holidays.Portugal`, que respeta los suspendidos en 2013–14) el 25-dic cae a 0.74/0.76/**0.67** (2012/13/**14**) y el 1-ene a 0.77–0.78 del mismo día de la semana previo; el 24 y el 31-dic casi no cambian. **Covariables futuras:** hora, día de la semana y mes en seno/coseno, más `is_holiday` e `is_weekend` (8). **Ventanas** (L=168, H=24, stride=1, asignadas por la partición del objetivo): 21,697 / 2,185 / 2,185 por serie (2.17 M / 218.5 k / 218.5 k en el panel) para train/val/test; `X:(B,168,9)`, `X_fut:(B,24,8)`, `y:(B,24)`.

**Auditoría de fugas** (cada punto es un `assert` en `run_00_eda.py`). (1) *Split aleatorio:* las particiones son temporales: los objetivos de train terminan el 30-jun-2014 y los de val cubren jul–sep; solo se barajan ventanas dentro de train. (2) *Escalado con toda la serie:* μ y σ salen de `Y[:, :val]`; multiplicar val+test ×10 no cambia el scaler (con la serie completa μ se habría movido 5.2 kWh en promedio). (3) *Early stopping con prueba:* el entrenamiento solo ve los 92 orígenes de validación; la prueba se usa una vez en `backtest`. (4) *Covariables mal alineadas:* `X_fut` es el calendario exacto de [t, t+24) y poner `y[t:]=999` no cambia X ni los rezagos de LightGBM. Además, `backtest(modelo)` entrega `Y[:, :o]` y, para las librerías, verifica `cutoff = o−1`.

## 2. Investigación (resumen; versión completa en el notebook)

**nn.LSTM/GRU** (`input_size, hidden_size, num_layers, batch_first, dropout`): `output (B,L,hidden)` contiene los estados de la última capa, `h_n/c_n (layers,B,hidden)` el último estado y la celda (la GRU no tiene `c_n`). **Conv1d causal:** se rellena a la izquierda con (k−1)·d; con dilataciones 2^i el campo receptivo es 1+(k−1)(2^n−1), por ejemplo k=2, n=8 → 256 ≥ 168. **TransformerEncoderLayer** (`d_model, nhead, dim_feedforward`): la atención no conoce el orden, así que se suma una codificación posicional, sinusoidal o aprendida (PatchTST usa parches de 24 h). **Nixtla:** formato largo `unique_id, ds, y`; `cross_validation(n_windows=92, step_size=24, refit=False)` reproduce este protocolo entrenando una sola vez. **Chronos** `predict_quantiles(context, prediction_length, quantile_levels)` → `(n, H, q)` y la media, zero-shot. **Local vs global:** un modelo por serie frente a uno compartido con normalización por serie. **Teacher forcing:** entrenar alimentando el valor real produce *exposure bias* en inferencia. **RevIN** normaliza cada ventana y revierte la salida, lo que da robustez a cambios de nivel. **Pinball** max(q·e, (q−1)·e): su minimizador es el cuantil q. **WAPE** = Σ|e|/Σ|y| pondera por volumen; **MASE** = MAE/MAE(seasonal naive m=24 en train) por serie y luego promediado, así que cada cliente pesa igual y no explota con y≈0, a diferencia del MAPE.

## 3. Protocolo y modelos

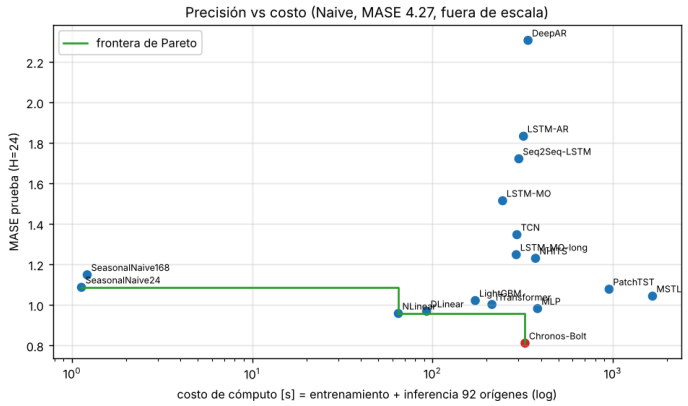
`backtest(modelo)`: 92 orígenes a las 00:00 (1-oct a 31-dic-2014); cada modelo recibe datos hasta las 23:00 del día previo y predice 24 h. Los pesos se entrenan una vez (train hasta 30-jun, early stopping/selección en val hasta 30-sep). Métricas en kWh agregadas sobre clientes y orígenes, además de por cliente, por paso, por hora y en la semana 22–28 dic (en `results/bt/`). Como el origen es siempre 00:00, *paso h* y *hora h−1* coinciden. **MAPE:** se excluyen los y=0; con clientes de consumo casi nulo un error pequeño en kWh da porcentajes enormes, por eso se reportan WAPE y MASE. **Modelos:** Naive, SeasonalNaive 24/168 y MSTL(24,168) con StatsForecast (locales, reajustados en cada origen con la historia previa; MSTL con las 2 semanas previas porque cuesta ~20–35 s por origen); LightGBM global (rezagos 1–24 y 168 + calendario + id, objetivo L1, recursivo); en PyTorch, MLP multi-salida, DLinear, NLinear, LSTM (autorregresiva y multi-salida) y Seq2Seq-LSTM con calendario en el decoder (AdamW, pérdida L1, *clip* 1.0, early stopping por MASE de val); en NeuralForecast, N-HiTS, TCN, PatchTST, iTransformer y DeepAR (StudentT); y Chronos-Bolt-small zero-shot. **Presupuesto en CPU:** los modelos propios muestrean 8k–40k ventanas por epoch y los de NeuralForecast usan 150 pasos (N-HiTS, TCN y DeepAR con el z-score por serie del pipeline y `scaler_type=identity`; el escalado por ventana de NF hacía divergir las covariables casi constantes), así que los modelos más pesados (LSTM, Transformers) están sub-entrenados respecto a una GPU.

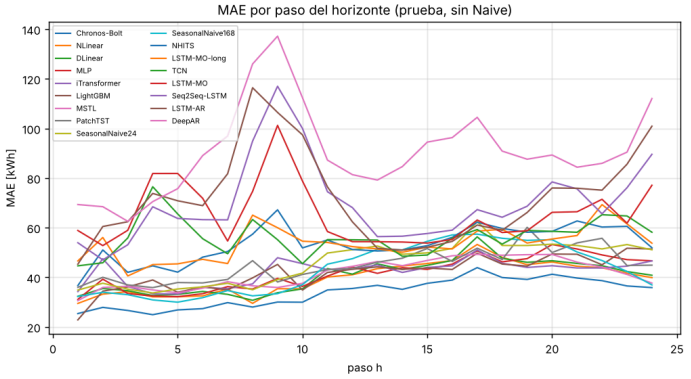
## 4. Resultados en prueba (92 orígenes, H=24, 100 clientes; ordenados por MASE)

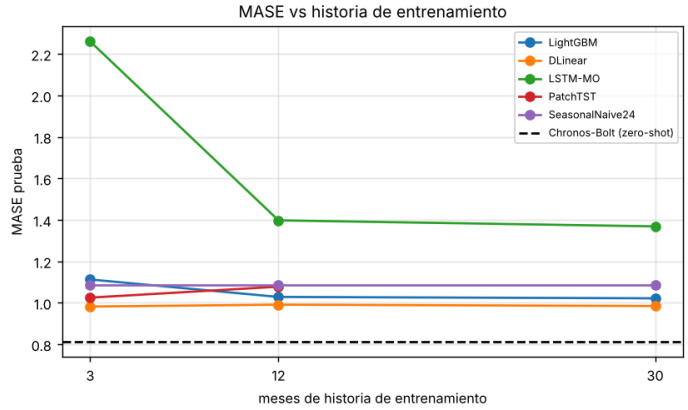
| Modelo | Familia | L | Estrategia | Cov. | Params | Entren. s | Infer. s | MAE | RMSE | WAPE | MASE | MASE H168 | MASE Nav. | Pinball | Cob. P10–P90 | Δ vs SN24 % |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Chronos-Bolt | Fundacional | 512 | multi-salida (cuantiles) | no | 47.72M | 0 | 326.4 | 33.8 | 205.0 | 0.063 | 0.812 | - | 1.597 | 11.7 | 0.80 | +25.4 |
| NLinear | Lineal | 168 | multi-salida | no | 4,056 | 64 | 0.8 | 39.7 | 190.0 | 0.074 | 0.958 | 1.106 | 2.009 | - | - | +11.9 |
| DLinear | Lineal | 168 | multi-salida | no | 8,112 | 92 | 1.1 | 40.5 | 191.7 | 0.075 | 0.969 | 1.096 | 2.037 | - | - | +10.9 |
| MLP | MLP | 168 | multi-salida | sí | 1.15M | 382 | 0.9 | 41.7 | 187.3 | 0.077 | 0.983 | 1.384 | 1.704 | - | - | +9.7 |
| iTransformer | Atención | 168 | multi-salida | no | 1.21M | 214 | 0.0 | 42.2 | 184.6 | 0.078 | 1.003 | - | 1.947 | - | - | +7.8 |
| LightGBM | Árboles | 168 | recursiva | sí | 261 | 160 | 12.6 | 41.8 | 185.6 | 0.078 | 1.022 | - | 1.934 | - | - | +6.0 |

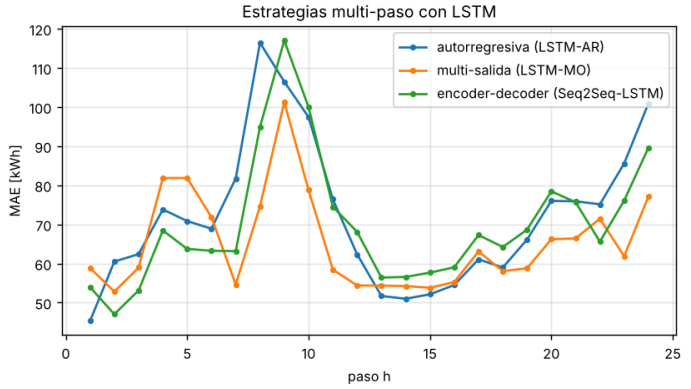
| Modelo | Familia | L | Estrategia | Cov. | Params | Entren. s | Infer. s | MAE | RMSE | WAPE | MASE | MASE H168 | MASE Nav. | Pinball | Cob. P10–P90 | Δ vs SN24 % |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| MSTL | Baseline | 336 | local (directo) | no | - | 0 | 1662.8 | 41.5 | 190.5 | 0.077 | 1.044 | - | 2.053 | - | - | +4.0 |
| PatchTST | Atención | 168 | multi-salida | no | 123,994 | 955 | 0.0 | 44.9 | 200.9 | 0.083 | 1.078 | - | 2.124 | - | - | +0.8 |
| SeasonalNaive24 | Baseline | 48 | local (directo) | no | - | 0 | 1.1 | 45.9 | 236.3 | 0.085 | 1.088 | 1.256 | 2.472 | - | - | +0.0 |
| SeasonalNaive168 | Baseline | 336 | local (directo) | no | - | 0 | 1.2 | 43.2 | 192.5 | 0.080 | 1.149 | 1.154 | 1.667 | - | - | -5.7 |
| NHITS | Residual (MLP jerárquico) | 168 | multi-salida | sí | 4.39M | 373 | 0.0 | 53.4 | 220.2 | 0.099 | 1.231 | - | 2.090 | - | - | -13.2 |
| LSTM-MO-long | Recurrente | 168 | multi-salida | sí | 37,208 | 290 | 2.1 | 53.6 | 213.3 | 0.099 | 1.249 | - | 2.144 | - | - | -14.8 |
| TCN | Convolucional | 168 | multi-salida (MQ) | sí | 47,769 | 294 | 0.0 | 56.4 | 207.5 | 0.104 | 1.348 | - | 2.146 | - | - | -23.9 |
| LSTM-MO | Recurrente | 168 | multi-salida | sí | 37,208 | 228 | 16.6 | 65.4 | 231.9 | 0.121 | 1.515 | - | 2.359 | - | - | -39.3 |
| Seq2Seq-LSTM | Encoder-decoder | 168 | encoder-decoder | sí | 38,209 | 286 | 15.1 | 70.1 | 241.5 | 0.130 | 1.722 | - | 2.594 | - | - | -58.4 |
| LSTM-AR | Recurrente | 168 | autorregresiva | sí | 19,265 | 294 | 26.0 | 72.2 | 243.3 | 0.134 | 1.834 | - | 2.615 | - | - | -68.6 |
| DeepAR | Probabilístico | 168 | autorregresiva (muestreo) | sí | 52,678 | 339 | 0.0 | 90.7 | 282.5 | 0.168 | 2.307 | - | 2.977 | 31.2 | 0.56 | -112.1 |
| Naive | Baseline | 24 | local (directo) | no | - | 0 | 0.8 | 174.4 | 531.8 | 0.323 | 4.273 | 4.311 | 4.350 | - | - | -293.0 |

Δ vs SN24: mejora porcentual del MASE frente al seasonal naive de 24 h. MASE Nav.: semana del 22 al 28 dic. Pinball/cobertura: solo modelos probabilísticos (cuantiles 0.1/0.5/0.9). Costo medido en CPU de 2 núcleos (sin GPU). LightGBM: "Params" = número de árboles. Modelos NeuralForecast: 150 pasos, 12 meses de historia (TCN y DeepAR: 6 meses, por memoria); su tiempo incluye la inferencia de `cross_validation`.



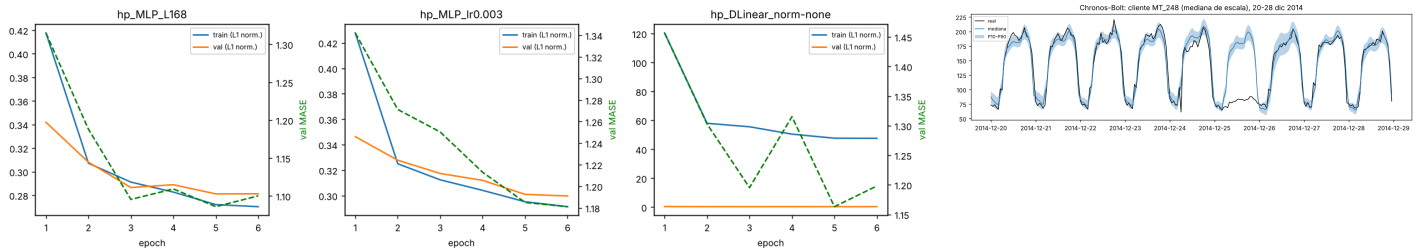




## Iteraciones de hiperparámetros (selección solo con validación; base: MLP L=168, 2×512, dropout 0.1, lr 1e-3, batch 256, z-score por serie, 30 meses)

| cambio | L | norm | lr | batch | params | epochs | s/epoch | train s | infer s | mem MB* | val_MASE | test_MASE |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| DLinear_L168 | 168 | series | 0.003 | 256 | 8112 | 6 | 1.04 | 7.8 | 0.21 | 581 | 0.9105 | 0.9849 |
| DLinear_L336 | 336 | series | 0.003 | 256 | 16176 | 6 | 1.25 | 9.6 | 0.36 | 610 | 0.9126 | 0.9718 |
| DLinear_L48 | 48 | series | 0.003 | 256 | 2352 | 6 | 0.78 | 5.3 | 0.06 | 558 | 0.9834 | 1.0758 |
| DLinear_norm-global | 168 | global | 0.003 | 256 | 8112 | 6 | 1.02 | 7.5 | 0.25 | 654 | 1.8481 | 1.9248 |
| DLinear_norm-none | 168 | none | 0.003 | 256 | 8112 | 6 | 1.12 | 8.3 | 0.30 | 654 | 1.1631 | 1.1936 |
| DLinear_norm-revin | 168 | revin | 0.003 | 256 | 8114 | 6 | 1.32 | 10.1 | 0.27 | 655 | 0.9681 | 1.0532 |
| DLinear_norm-series | 168 | series | 0.003 | 256 | 8112 | 6 | 1.03 | 8.0 | 0.16 | 654 | 0.9105 | 0.9849 |
| MLP_L168 | 168 | series | 0.001 | 256 | 1147928 | 6 | 3.21 | 23.8 | 0.64 | 610 | 1.0859 | 1.1101 |
| MLP_L336 | 336 | series | 0.001 | 256 | 1922072 | 6 | 4.24 | 31.6 | 1.11 | 654 | 1.1229 | 1.1200 |
| MLP_L48 | 48 | series | 0.001 | 256 | 594968 | 6 | 2.08 | 14.5 | 0.27 | 579 | 1.1193 | 1.1912 |
| MLP_bs1024 | 168 | series | 0.001 | 1024 | 1147928 | 6 | 2.32 | 17.6 | 0.66 | 670 | 1.1435 | 1.2124 |
| MLP_dropout0.3 | 168 | series | 0.001 | 256 | 1147928 | 5 | 3.21 | 19.6 | 0.72 | 663 | 1.2054 | 1.2455 |
| MLP_hidden128 | 168 | series | 0.001 | 256 | 237848 | 6 | 1.36 | 9.8 | 0.32 | 655 | 1.0963 | 1.2013 |
| MLP_layers1 | 168 | series | 0.001 | 256 | 885272 | 6 | 2.23 | 16.4 | 0.50 | 656 | 0.9848 | 1.0783 |
| MLP_lr0.003 | 168 | series | 0.003 | 256 | 1147928 | 6 | 3.02 | 22.2 | 0.65 | 663 | 1.1811 | 1.2289 |

| cambio | L | norm | lr | batch | params | epochs | s/epoch | train s | infer s | mem MB* | val_MASE | test_MASE |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| MLP_norm-none | 168 | none | 0.001 | 256 | 1147928 | 6 | 3.02 | 22.1 | 0.68 | 655 | 1.4908 | 1.4963 |
| MLP_norm-revin | 168 | revin | 0.001 | 256 | 1147930 | 6 | 3.44 | 25.6 | 0.68 | 655 | 1.3313 | 1.2792 |

*Memoria pico: RSS de CPU del proceso (no hay GPU). Cada fila registra pérdida de train/val, MAE y MASE de validación por epoch en `results/experiments/*.json`. Prueba = una sola evaluación por configuración, reportada para el análisis y no usada para elegir.




### Local vs. global (NLinear, 10 clientes) · semillas · reentrenamiento

| unique_id | mean_kwh | MASE_local | MASE_global |
|---|---|---|---|
| MT_046 | 131.09 | 0.86 | 0.86 |
| MT_051 | 54.62 | 0.78 | 0.75 |
| MT_084 | 211.34 | 0.81 | 0.97 |
| MT_187 | 57.34 | 1.48 | 1.40 |
| MT_250 | 92.03 | 0.96 | 0.93 |
| MT_274 | 322.40 | 0.88 | 0.86 |
| MT_280 | 284.98 | 1.12 | 1.07 |
| MT_318 | 168.24 | 1.07 | 0.90 |
| MT_349 | 187.53 | 0.99 | 0.96 |
| MT_360 | 512.05 | 0.94 | 0.88 |

**Semillas (NLinear, 0/1/2):** MASE de prueba 0.9592 / 0.9597 / 0.9594, σ = 0.00025.

**Reentrenamiento mensual** (ventana expandible, 3 orígenes: 1-oct, 1-nov, 1-dic): NLinear 0.9585 → 0.9634, sin mejora. El mejor baseline (MSTL, 1.044) ya se reajusta en cada origen, así que su resultado es idéntico.

**Estrategias LSTM** (mismo presupuesto): MAE h=1 → h=24: autorregresiva 45 → 101, multi-salida 59 → 77, encoder–decoder 54 → 90 kWh.

**Global vs local (promedio):** 0.957 vs 0.989; el global gana en 8 de 10 clientes.

## 5. Discusión y conclusiones

**¿Qué modelo ganó? Chronos-Bolt-small zero-shot** obtuvo MASE **0.812** (MAE 33.8 kWh, WAPE 6.3%), un **25.4% mejor que el seasonal naive de 24 h** (MASE 1.088). Entre los modelos entrenados el mejor fue **NLinear (0.958, −11.9%)**, seguido de DLinear (0.969), MLP (0.983), iTransformer (1.003), LightGBM (1.022) y MSTL (1.044, el mejor baseline estadístico). NLinear entrena en ~1 min en CPU, tiene 4 k parámetros y predice los 92 orígenes en <1 s: en producción la mejora sí compensa su costo, porque reentrenarlo y mantenerlo es trivial. Chronos no requiere entrenamiento, pero la inferencia de los 92 orígenes costó ~5.4 min en CPU (frente a <1 s de NLinear) y depende de un modelo de 48 M parámetros. La frontera de Pareto (MASE vs costo total de entrenamiento + inferencia) queda formada por los seasonal naive, NLinear y Chronos.

**DLinear vs. Transformers.** Sí se replica: con H=24, DLinear (0.969) y NLinear (0.958) superan a los dos Transformers, iTransformer (1.003) y PatchTST (1.078), ambos entrenados con 12 meses y 150 pasos en CPU, y también a N-HiTS (1.231), TCN (1.348) y a los modelos recurrentes. Con H=168, DLinear (1.096) y NLinear (1.106) quedan cerca del seasonal naive semanal (SN168 ≈ 1.15–1.16) y muy por delante del MLP (1.384), cuya cantidad de parámetros crece con H. La ventaja de los lineales es robustez y bajo costo; los Transformers necesitarían bastante más cómputo para aprovechar su capacidad.

**Estrategias multi-paso (LSTM, mismo presupuesto).** La multi-salida fue la mejor (1.515), seguida del encoder–decoder (1.722) y la autorregresiva (1.834). La **acumulación de errores de la autorregresiva** es clara: su MAE pasa de 45 kWh en h=1 (la mejor de las tres, porque se entrena como modelo de un paso) a 101 kWh en h=24 (×2.2). La multi-salida pasa de 59 a 77 kWh (×1.3) y el seq2seq de 54 a 90. Por familias: el seasonal naive y los lineales tienen error casi plano en h (el paso coincide con la hora del día y su error sigue el perfil diario); LightGBM recursivo crece de 23 a 51 kWh; Chronos va de 25 a 36. Con más presupuesto (LSTM-MO-long, 12 epochs) la LSTM bajó a 1.249, todavía detrás de los lineales: en CPU las recurrentes quedan sub-entrenadas.

**Ventana L, normalización e historia** (selección por MASE de validación, tabla de iteraciones). *L:* en DLinear, L=48 da 0.983 en validación frente a 0.911 con L=168 y 0.913 con L=336. Hace falta ver al menos una semana para capturar el ciclo semanal, y duplicarla no aporta. El MLP prefiere L=168 (1.086). *Normalización:* es el factor más importante. En DLinear, z-score por serie (0.911) < RevIN (0.968) < sin normalizar (1.163) ≪ global (1.848). Con escalas de 8 a 11,000 kWh, la normalización global hace que los clientes grandes dominen la pérdida. En el MLP, sin normalizar da 1.491 y RevIN 1.331, frente a 1.086 por serie. *Historia:* DLinear casi no cambia con 3, 12 o 30 meses (0.982 / 0.991 / 0.985 en prueba), porque con 8 k parámetros 3 meses ya bastan. LightGBM sí mejora con más historia (1.113 → 1.029 → 1.022). La LSTM la necesita (3 m: 2.26 → 12 m: 1.40 → 30 m: 1.37) y PatchTST empeoró de 3 a 12 meses (1.025 → 1.078) con el mismo número de pasos: más datos con presupuesto fijo significa menos pasadas por ejemplo. En estos datos el deep learning lineal ya supera a LightGBM y al seasonal naive **desde 3 meses**; los modelos recurrentes no lo logran ni con 30 meses bajo este presupuesto. *Otros (MLP):* 1 capa (0.985) fue mejor que 2 (1.086); más dropout (0.3: 1.205), lr 3e-3 (1.181), oculto 128 (1.096) y batch 1024 (1.144) empeoraron. *¿Una pérdida de entrenamiento más baja implica mejor MASE de validación?* No siempre: en las curvas, la pérdida de entrenamiento baja de forma monótona mientras el MASE de validación oscila, y el MLP de 2 capas tiene menor pérdida de entrenamiento que el de 1 capa pero peor validación (sobreajuste). Por eso el early stopping se hace con el MASE de validación.

**Local vs. global** (NLinear, 10 clientes elegidos con semilla). El global gana en promedio: MASE 0.957 frente a 0.989 local, y es mejor en 8 de 10 clientes. Los que más se benefician (MT_318: −0.18; MT_187: −0.07) tienen perfiles irregulares, donde compartir patrones de calendario entre series regulariza. Solo MT_084 mejora con su modelo propio (−0.16), lo que sugiere un patrón idiosincrático. Además, un modelo global se entrena una vez, en lugar de 10 o 100.

**Chronos zero-shot.** Fue el mejor en MASE, MAE y WAPE sin ver ningún dato de entrenamiento, y su intervalo P10–P90 cubre el **80.4%** (nominal 80%) con pinball 11.7 kWh. Lo usaría cuando hay poca historia, series nuevas (*cold start*), muchos clientes heterogéneos o no hay tiempo ni infraestructura para entrenar. Entrenaría una red ligera (NLinear) cuando la latencia o el costo de inferencia importan, o cuando hay covariables que Chronos-Bolt no usa (feriados).

**Pronóstico probabilístico.** Chronos-Bolt (pinball medio 11.7 kWh, cobertura P10–P90 de 80.4%) está prácticamente calibrado. DeepAR, con solo 150 pasos y 6 meses de historia en CPU, quedó sub-entrenado: MASE 2.307, pinball 31.2 kWh y cobertura de 55.7%. Sus intervalos son demasiado estrechos respecto a su error.

**Navidad y Año Nuevo** (22–28 dic). Todos los modelos empeoran: el MASE sube a 1.6–2.6. Los mejores esa semana fueron Chronos (1.597), SN168 (1.667) y el MLP (1.704); NLinear y DLinear suben a ~2.0–2.04, y LightGBM, que tiene la covariable de feriado, a 1.934. Solo hay un feriado en la semana y es irrepetible en entrenamiento (2 ocurrencias en 30 meses), así que la covariable apenas ayuda. Los modelos que usan la semana previa (SN168) o mucho contexto (Chronos, MLP) degradan menos. Es un cambio de régimen que afecta a todos.

**Semillas.** NLinear con semillas 0, 1 y 2 dio MASE de prueba 0.9592 / 0.9597 / 0.9594, con $\boldsymbol{\sigma = 0.00025}$: el modelo lineal es prácticamente determinista. Las diferencias de ~0.01 entre modelos lineales son mayores que la varianza por semilla.

**Reentrenamiento mensual** (ventana expandible, 1-oct / 1-nov / 1-dic, epochs fijos = mejor epoch de validación). NLinear pasa de 0.9585 (entrenado una vez) a 0.9634: **no mejora**. Tres meses adicionales sobre 30 de historia no cambian un modelo lineal. Para el mejor baseline (**MSTL**, MASE 1.044) el reentrenamiento mensual es idéntico por construcción: es un modelo local que ya se reajusta en cada origen diario con la historia previa (una versión más agresiva del reentrenamiento), así que su error no cambia.

**Limitaciones.** Todo corrió en una CPU de 2 núcleos y sin GPU, con 100 clientes. Los modelos pesados (LSTM, Transformers, DeepAR) tuvieron presupuestos reducidos, y los de NeuralForecast se entrenaron con 12 meses de historia por memoria (TCN y DeepAR con 6 meses, tras dos cierres por falta de memoria). Sus resultados son cotas inferiores de lo que lograrían en GPU. MSTL se ajustó con una ventana de 2 semanas por origen debido a su costo (~20–35 s por origen).

**Repositorio:** [agregar enlace al repositorio de GitHub]